

고객을 세그먼테이션하자 [프로젝트] - 이신민

11-2. 데이터 불러오기

데이터 살펴보기

- 테이블에 있는 10개의 행만 출력하기

```
SELECT *  
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.data`  
LIMIT 10;
```

작업 정보		결과	시각화	JSON	실행 세부정보	실행 그래프		
행	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
1	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00 UTC	2.55	17850	United Kingdom
2	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom
3	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00 UTC	2.75	17850	United Kingdom
4	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom
5	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom
6	536365	22752	SET 7 BABUSHKA NESTING BOXES	2	2010-12-01 08:26:00 UTC	7.65	17850	United Kingdom
7	536365	21730	GLASS STAR FROSTED T-LIGHT HOLDER	6	2010-12-01 08:26:00 UTC	4.25	17850	United Kingdom
8	536366	22633	HAND WARMER UNION JACK	6	2010-12-01 08:28:00 UTC	1.85	17850	United Kingdom
9	536366	22632	HAND WARMER RED POLKA DOT	6	2010-12-01 08:28:00 UTC	1.85	17850	United Kingdom
10	536367	84879	ASSORTED COLOUR BIRD ORNAMENT	32	2010-12-01 08:34:00 UTC	1.69	13047	United Kingdom

- 전체 데이터는 몇 행으로 구성되어 있는지 확인하기

```
SELECT COUNT(*) AS row_count  
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.data`;
```

행	row_count
1	541909

데이터 수 세기

- COUNT 함수를 사용해서, 각 컬럼별 데이터 포인트의 수를 세어 보기

```
SELECT
  COUNT(InvoiceNo) AS COUNT_InvoiceNo,
  COUNT(StockCode) AS COUNT_StockCode,
  COUNT(Description) AS COUNT_Description,
  COUNT(Quantity) AS COUNT_Quantity,
  COUNT(InvoiceDate) AS COUNT_InvoiceDate,
  COUNT(UnitPrice) AS COUNT_UnitPrice,
  COUNT(CustomerID) AS COUNT_CustomerID,
  COUNT(Country) AS COUNT_Country
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.data`;
```

COUNT_InvoiceNo	COUNT_StockCode	COUNT_Description	COUNT_Quantity	COUNT_InvoiceDate	COUNT_UnitPrice	COUNT_CustomerID	COUNT_Country
541909	541909	540455	541909	541909	541909	406829	541909

11-4. 데이터 전처리 방법(1): 결측치 제거

컬럼 별 누락된 값의 비율 계산

- 각 컬럼 별 누락된 값의 비율을 계산
 - 각 컬럼에 대해서 누락 값을 계산한 후, 계산된 누락 값을 UNION ALL을 통해 합치기

```
SELECT
  'InvoiceNo' AS column_name,
  ROUND(SUM(CASE WHEN InvoiceNo IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_percentage
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.data`

UNION ALL
```

```
SELECT
    'StockCode',
    ROUND(SUM(CASE WHEN StockCode IS NULL THEN 1 ELSE 0 EN
D) / COUNT(*) * 100, 2)
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.da
ta`
```

```
UNION ALL
```

```
SELECT
    'Description',
    ROUND(SUM(CASE WHEN Description IS NULL THEN 1 ELSE 0
END) / COUNT(*) * 100, 2)
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.da
ta`
```

```
UNION ALL
```

```
SELECT
    'Quantity',
    ROUND(SUM(CASE WHEN Quantity IS NULL THEN 1 ELSE 0 EN
D) / COUNT(*) * 100, 2)
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.da
ta`
```

```
UNION ALL
```

```
SELECT
    'InvoiceDate',
    ROUND(SUM(CASE WHEN InvoiceDate IS NULL THEN 1 ELSE 0
END) / COUNT(*) * 100, 2)
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.da
ta`
```

```
UNION ALL
```

```
SELECT
```

```

    'UnitPrice',
    ROUND(SUM(CASE WHEN UnitPrice IS NULL THEN 1 ELSE 0 EN
D) / COUNT(*) * 100, 2)
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.da
ta`

UNION ALL

SELECT
    'CustomerID',
    ROUND(SUM(CASE WHEN CustomerID IS NULL THEN 1 ELSE 0 E
ND) / COUNT(*) * 100, 2)
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.da
ta`

UNION ALL

SELECT
    'Country',
    ROUND(SUM(CASE WHEN Country IS NULL THEN 1 ELSE 0 END)
/ COUNT(*) * 100, 2)
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.da
ta`;

```

행	column_name ▼	missing_percentage ▼
1	InvoiceNo	0.0
2	StockCode	0.0
3	Description	0.27
4	Quantity	0.0
5	InvoiceDate	0.0
6	UnitPrice	0.0
7	CustomerID	24.93
8	Country	0.0

결측치 처리 전략

- `StockCode = '85123A'` 의 `Description` 을 추출하는 쿼리문을 작성하기

```
SELECT DISTINCT Description
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.data`
WHERE StockCode = '85123A';
```

행	Description
1	WHITE HANGING HEART T-LIG...
2	?
3	wrongly marked carton 22804
4	CREAM HANGING HEART T-LIG...

결측치 처리

- `DELETE` 구문을 사용하며, `WHERE` 절을 통해 데이터를 제거할 조건을 제시

```
DELETE FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.data`
WHERE CustomerID IS NULL
OR Description IS NULL;
```

i 이 문으로 data의 행 135,080개가 삭제되었습니다.

11-5. 데이터 전처리(2): 중복값 처리

중복값 확인

- 중복된 행의 수를 세어보기
 - 8개의 컬럼에 그룹 함수를 적용한 후, COUNT가 1보다 큰 데이터를 세어보기

```
SELECT COUNT(*) AS duplicate_rows
FROM (
    SELECT
        InvoiceNo,
        StockCode,
        Description,
        Quantity,
        InvoiceDate,
        UnitPrice,
        CustomerID,
        Country,
        COUNT(*) AS cnt
    FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.
data`
    GROUP BY
        InvoiceNo,
        StockCode,
        Description,
        Quantity,
        InvoiceDate,
        UnitPrice,
        CustomerID,
        Country
    HAVING COUNT(*) > 1
);
```

행	duplicate_rows
1	4837

중복값 처리

- 중복값을 제거하는 쿼리문 작성하기
 - CREATE OR REPLACE TABLE 구문을 활용하여 모든 컬럼(*)을 DISTINCT 한 데이터로 업데이트

```
CREATE OR REPLACE TABLE `project-1799b89a-43b7-4dae-ac9.modulabs_project.data` AS
SELECT DISTINCT *
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.data`;
```



이 문으로 이름이 data인 테이블이 교체되었습니다.

11-6. 데이터 전처리(3): 오류값 처리

InvoiceNo 살펴보기

- 고유(unique)한 InvoiceNo의 개수를 출력하기

```
SELECT COUNT(DISTINCT InvoiceNo) AS unique_invoice_count
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.data`;
```

행	unique_invoice_c...
1	22190

- 고유한 InvoiceNo를 앞에서부터 100개를 출력하기

```
SELECT DISTINCT InvoiceNo
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.data`
LIMIT 100;
```

행	InvoiceNo
1	541431
2	C541433
3	537626
4	542237
5	549222
6	556201
7	562032
8	573511
9	581180
10	539318

- **InvoiceNo** 가 'C'로 시작하는 행을 필터링 할 수 있는 쿼리문을 작성하기 (100행까지만 출력)

```
SELECT *
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.data`
WHERE InvoiceNo LIKE 'C%'
LIMIT 100;
```


행	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
1	C541433	23166	MEDIUM CERAMIC TOP STORAGE JAR	-74215	2011-01-18 10:17:00 UTC	1.04	12346	United Kingdom
2	C545329	M	Manual	-1	2011-03-01 15:47:00 UTC	280.05	12352	Norway
3	C545329	M	Manual	-1	2011-03-01 15:47:00 UTC	183.75	12352	Norway
4	C545330	M	Manual	-1	2011-03-01 15:49:00 UTC	376.5	12352	Norway
5	C547388	37448	CERAMIC CAKE DESIGN SPOTTED MUG	-12	2011-03-22 16:07:00 UTC	1.49	12352	Norway
6	C547388	84050	PINK HEART SHAPE EGG FRYING PAN	-12	2011-03-22 16:07:00 UTC	1.65	12352	Norway
7	C547388	22784	LANTERN CREAM GAZEBO	-3	2011-03-22 16:07:00 UTC	4.95	12352	Norway
8	C547388	21914	BLUE HARMONICA IN BOX	-12	2011-03-22 16:07:00 UTC	1.25	12352	Norway
9	C547388	22701	PINK DOG BOWL	-6	2011-03-22 16:07:00 UTC	2.95	12352	Norway

- 구매 건 상태가 **Canceled** 인 데이터의 비율(%) - 소수점 첫번째 자리까지

```
SELECT
  ROUND(
    SUM(CASE WHEN InvoiceNo LIKE 'C%' THEN 1 ELSE 0 END)
    / COUNT(*) * 100,
    1
  ) AS canceled_ratio_percent
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.data`;
```

행	canceled_ratio_percent
1	2.2

StockCode 살펴보기

- 고유한 **StockCode** 의 개수를 출력하기

```
SELECT COUNT(DISTINCT StockCode) AS unique_stockcode_count
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.data`;
```

행	unique_stockcode_count
1	3684

- 어떤 제품이 가장 많이 판매되었는지 보기 위하여 **StockCode** 별 등장 빈도를 출력하기
 - 상위 10개의 제품들을 출력하기

```
SELECT StockCode, COUNT(*) AS sell_cnt
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.data`
GROUP BY StockCode
ORDER BY sell_cnt DESC
LIMIT 10;
```

행	StockCode	sell_cnt
1	85123A	2065
2	22423	1894
3	85099B	1659
4	47566	1409
5	84879	1405
6	20725	1346
7	22720	1224
8	POST	1196
9	22197	1110
10	23203	1108

- StockCode**의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고

- 숫자가 0~1개인 값들에는 어떤 코드들이 들어가 있는지 출력하기

```
SELECT DISTINCT StockCode, number_count
FROM (
  SELECT
    StockCode,
    LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode,
    r'[0-9]', '')) AS number_count
  FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.
  data`
)
WHERE number_count <= 1;
```

행	StockCode ▼	number_count ▼
1	POST	0
2	M	0
3	C2	1
4	D	0
5	BANK CHARGES	0
6	PADS	0
7	DOT	0
8	CRUK	0

- StockCode**의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고
 - 숫자가 0~1개인 값들을 가지고 있는 데이터 수는 전체 데이터 수 대비 몇 퍼센트인지 구하기 (소수점 두 번째 자리까지)

```
SELECT
  ROUND(
    COUNT(*) / (
```

```

SELECT COUNT(*)
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.data`
) * 100,
2
) AS percentage
FROM (
SELECT StockCode
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.data`
WHERE LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) <= 1
);

```

행	percentage
1	0.48

- 제품과 관련되지 않은 거래 기록을 제거하기

```

DELETE FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.data`
WHERE StockCode IN (
SELECT DISTINCT StockCode
FROM (
SELECT
StockCode,
LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.data`
)
WHERE number_count <= 1
);

```

i 이 문으로 data의 행 1,915개가 삭제되었습니다.

Description 살펴보기

- 고유한 Description 별 출현 빈도를 계산하고 상위 30개를 출력하기

```
SELECT
  Description,
  COUNT(*) AS description_count
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.data`
GROUP BY Description
ORDER BY description_count DESC
LIMIT 30;
```

행	Description ▼	description_count ▼
1	WHITE HANGING HEART T-LIG...	2058
2	REGENCY CAKESTAND 3 TIER	1894
3	JUMBO BAG RED RETROSPOT	1659
4	PARTY BUNTING	1409
5	ASSORTED COLOUR BIRD ORNA...	1405
6	LUNCH BAG RED RETROSPOT	1345
7	SET OF 3 CAKE TINS PANTRY D...	1224
8	LUNCH BAG BLACK SKULL.	1099
9	PACK OF 72 RETROSPOT CAKE ...	1062
10	SPOTTY BUNTING	1026

- 서비스 관련 정보를 포함하는 행들을 제거하기

```
DELETE
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.data`
WHERE Description IN ('High Resolution Image', 'Next Day Carriage');
```

 이 문으로 data의 행 83개가 삭제되었습니다.

- 대소문자를 혼합하고 있는 데이터를 대문자로 표준화 하기

```
CREATE OR REPLACE TABLE `project-1799b89a-43b7-4dae-ac9.modulabs_project.data` AS
SELECT
    * EXCEPT (Description),
    UPPER(Description) AS Description
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.data`;
```

 이 문으로 이름이 data인 테이블이 교체되었습니다.

UnitPrice 살펴보기

- UnitPrice 의 최솟값, 최댓값, 평균을 구하기

```
SELECT
  MIN(UnitPrice) AS min_price,
  MAX(UnitPrice) AS max_price,
  AVG(UnitPrice) AS avg_price
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.data`;
```

행	min_price ▼	max_price ▼	avg_price ▼
1	0.0	649.5	2.904956757406...

- 단가가 0원인 거래의 개수, 구매 수량(**Quantity**)의 최솟값, 최댓값, 평균 구하기

```
SELECT
  COUNT(*) AS cnt_quantity,
  MIN(Quantity) AS min_quantity,
  MAX(Quantity) AS max_quantity,
  AVG(Quantity) AS avg_quantity
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.data`
WHERE UnitPrice = 0;
```

행	cnt_quantity ▼	min_quantity ▼	max_quantity ▼	avg_quantity ▼
1	33	1	12540	420.5151515151...

- **UnitPrice = 0** 를 제거하고 일관된 데이터셋을 유지하기

```
CREATE OR REPLACE TABLE `project-1799b89a-43b7-4dae-ac9.modulabs_project.data` AS
SELECT *
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.data`
WHERE UnitPrice > 0;
```

```
ta`
WHERE UnitPrice != 0;
```

 이 문으로 이름이 data인 테이블이 교체되었습니다.

11-7. RFM 스코어

Recency

- **InvoiceDate** 컬럼을 연월일 자료형으로 변경하기

```
SELECT
    DATE(InvoiceDate) AS InvoiceDay,
    *
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.da
ta`;
```

행	InvoiceDay	InvoiceNo	StockCode	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	Description
1	2011-01-18	541431	23166	74215	2011-01-18 10:01:00 UTC	1.04	12346	United Kingdom	MEDIUM CERAMIC TOP STORA...
2	2011-01-18	C541433	23166	-74215	2011-01-18 10:17:00 UTC	1.04	12346	United Kingdom	MEDIUM CERAMIC TOP STORA...
3	2010-12-07	537626	22195	12	2010-12-07 14:57:00 UTC	1.65	12347	Iceland	LARGE HEART MEASURING SPO...
4	2010-12-07	537626	22727	4	2010-12-07 14:57:00 UTC	3.75	12347	Iceland	ALARM CLOCK BAKELIKE RED
5	2010-12-07	537626	22729	4	2010-12-07 14:57:00 UTC	3.75	12347	Iceland	ALARM CLOCK BAKELIKE ORAN...
6	2010-12-07	537626	22728	4	2010-12-07 14:57:00 UTC	3.75	12347	Iceland	ALARM CLOCK BAKELIKE PINK
7	2010-12-07	537626	22492	36	2010-12-07 14:57:00 UTC	0.65	12347	Iceland	MINI PAINT SET VINTAGE
8	2010-12-07	537626	20780	12	2010-12-07 14:57:00 UTC	4.65	12347	Iceland	BLACK EAR MUFF HEADPHONES
9	2010-12-07	537626	21064	6	2010-12-07 14:57:00 UTC	5.95	12347	Iceland	BOOM BOX SPEAKER BOYS

- 가장 최근 구매 일자를 MAX() 함수로 찾아보기

```
SELECT
    MAX(DATE(InvoiceDate)) OVER() AS most_recent_date,
    DATE(InvoiceDate) AS InvoiceDay,
```


*

```
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.data`;
```

행	most_recent_date	InvoiceDay	InvoiceNo	StockCode	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	Description
1	2011-12-09	2011-01-18	541431	23166	74215	2011-01-18 10:01:00 UTC	1.04	12346	United Kingdom	MEDIUM CERAMIC TOP STORA...
2	2011-12-09	2011-01-18	C541433	23166	-74215	2011-01-18 10:17:00 UTC	1.04	12346	United Kingdom	MEDIUM CERAMIC TOP STORA...
3	2011-12-09	2010-12-07	537626	849978	6	2010-12-07 14:57:00 UTC	3.75	12347	Iceland	RED 3 PIECE RETROSPOT CUTL...
4	2011-12-09	2010-12-07	537626	22494	12	2010-12-07 14:57:00 UTC	1.25	12347	Iceland	EMERGENCY FIRST AID TIN
5	2011-12-09	2010-12-07	537626	21171	12	2010-12-07 14:57:00 UTC	1.45	12347	Iceland	BATHROOM METAL SIGN
6	2011-12-09	2010-12-07	537626	85116	12	2010-12-07 14:57:00 UTC	2.1	12347	Iceland	BLACK CANDELABRA T-LIGHT H...
7	2011-12-09	2010-12-07	537626	22774	12	2010-12-07 14:57:00 UTC	1.25	12347	Iceland	RED DRAWER KNOB ACRYLIC E...
8	2011-12-09	2010-12-07	537626	22195	12	2010-12-07 14:57:00 UTC	1.65	12347	Iceland	LARGE HEART MEASURING SPQ...
9	2011-12-09	2010-12-07	537626	22805	12	2010-12-07 14:57:00 UTC	1.25	12347	Iceland	BLUE DRAWER KNOB ACRYLIC ...

- 유저 별로 가장 큰 InvoiceDay를 찾아서 가장 최근 구매일로 저장하기

```
SELECT
  CustomerID,
  MAX(DATE(InvoiceDate)) AS InvoiceDay
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.data`
GROUP BY CustomerID;
```

행	CustomerID	InvoiceDay
1	12346	2011-01-18
2	12347	2011-12-07
3	12348	2011-09-25
4	12349	2011-11-21
5	12350	2011-02-02
6	12352	2011-11-03
7	12353	2011-05-19
8	12354	2011-04-21
9	12355	2011-05-09
10	12356	2011-11-17

- 가장 최근 일자(`most_recent_date`)와 유저별 마지막 구매일(`InvoiceDay`)간의 차이를 계산하기

```
SELECT
    CustomerID,
    EXTRACT(DAY FROM MAX(InvoiceDay) OVER() - InvoiceDay)
AS recency
FROM (
    SELECT
        CustomerID,
        MAX(DATE(InvoiceDate)) AS InvoiceDay
    FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.
data`
    GROUP BY CustomerID
);
```

행	CustomerID	recency
1	12531	14
2	12920	17
3	13014	2
4	13461	38
5	13476	178
6	13764	70
7	13841	252
8	14062	9
9	14271	225
10	14334	16

- 최종 데이터 셋에 필요한 데이터들을 각각 정제해서 이어붙이고 지금까지의 결과를 `user_r` 이라는 이름의 테이블로 저장하기

```
CREATE OR REPLACE TABLE `project-1799b89a-43b7-4dae-ac9.modulabs_project.user_r` AS
SELECT
  CustomerID,
  EXTRACT(DAY FROM MAX(InvoiceDay) OVER() - InvoiceDay)
AS recency
FROM (
  SELECT
    CustomerID,
    MAX(DATE(InvoiceDate)) AS InvoiceDay
  FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.data`
  GROUP BY CustomerID
);
```

이 문으로 이름이 `user_r`인 테이블이 교체되었습니다.

Frequency

- 고객마다 고유한 InvoiceNo의 수를 세어보기

```
SELECT
  CustomerID,
  COUNT(DISTINCT InvoiceNo) AS purchase_cnt
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.data`
GROUP BY CustomerID;
```

행	CustomerID ▼	purchase_cnt ▼
1	12346	2
2	12347	7
3	12348	4
4	12349	1
5	12350	1
6	12352	8
7	12353	1
8	12354	1

- 각 고객 별로 구매한 아이템의 총 수량 더하기

```
SELECT
  CustomerID,
  SUM(Quantity) AS item_cnt
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.data`
GROUP BY CustomerID;
```

행	CustomerID	item_cnt
1	12346	0
2	12347	2458
3	12348	2332
4	12349	630
5	12350	196
6	12352	463
7	12353	20
8	12354	530
9	12355	240
10	12356	1573

- 전체 거래 건수 계산과 구매한 아이템의 총 수량 계산의 결과를 합쳐서 `user_rf` 라는 이름의 테이블에 저장하기

```
CREATE OR REPLACE TABLE `project-1799b89a-43b7-4dae-ac9.modulabs_project.user_rf` AS
```

```
WITH purchase_cnt AS (
  SELECT
    CustomerID,
    COUNT(DISTINCT InvoiceNo) AS purchase_cnt
  FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.data`
  GROUP BY CustomerID
),
```

```
item_cnt AS (
  SELECT
    CustomerID,
    SUM(Quantity) AS item_cnt
  FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.
```

```

data`
  GROUP BY CustomerID
)

SELECT
  pc.CustomerID,
  pc.purchase_cnt,
  ic.item_cnt,
  ur.recency
FROM purchase_cnt AS pc
JOIN item_cnt AS ic
  ON pc.CustomerID = ic.CustomerID
JOIN `project-1799b89a-43b7-4dae-ac9.modulabs_project.us
er_r` AS ur
  ON pc.CustomerID = ur.CustomerID;

```

 이 문으로 이름이 user_rf인 테이블이 교체되었습니다.

Monetary

- 고객별 총 지출액 계산 (소수점 첫째 자리에서 반올림)

```

SELECT
  CustomerID,
  ROUND(SUM(Quantity * UnitPrice), 1) AS user_total
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.da
ta`
GROUP BY CustomerID;

```

행	CustomerID ▼	user_total ▼
1	12346	0.0
2	12347	4310.0
3	12348	1437.2
4	12349	1457.5
5	12350	294.4
6	12352	1265.4
7	12353	89.0
8	12354	1079.4
9	12355	459.4
10	12356	2487.4

- 고객별 평균 거래 금액 계산

- 고객별 평균 거래 금액을 구하기 위해 1) `data` 테이블을 `user_rf` 테이블과 조인 (LEFT JOIN) 한 후, 2) `purchase_cnt` 로 나누어서 3) `user_rfm` 테이블로 저장하기

```
CREATE OR REPLACE TABLE `project-1799b89a-43b7-4dae-ac9.
modulabs_project.user_rfm` AS
SELECT
  rf.CustomerID AS CustomerID,
  rf.purchase_cnt,
  rf.item_cnt,
  rf.recency,
  ut.user_total,
  ROUND(ut.user_total / rf.purchase_cnt, 1) AS user_aver
age
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.us
er_rf` rf
LEFT JOIN (
  SELECT
    CustomerID,
```

```

ROUND(SUM(Quantity * UnitPrice), 1) AS user_total
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.
data`
GROUP BY CustomerID
) ut
ON rf.CustomerID = ut.CustomerID;

```



이 문으로 이름이 user_rfm인 테이블이 교체되었습니다.

RFM 통합 테이블 출력하기

- 최종 user_rfm 테이블을 출력하기

```

SELECT *
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.us
er_rfm`;

```

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average
1	12713	1	505	0	794.5	794.5
2	13436	1	76	1	196.9	196.9
3	14569	1	79	1	227.4	227.4
4	13298	1	96	1	360.0	360.0
5	15520	1	314	1	343.5	343.5
6	15195	1	1404	2	3861.0	3861.0
7	15471	1	256	2	454.5	454.5
8	14204	1	72	2	150.6	150.6
9	17914	1	457	3	329.4	329.4

11-8. 추가 Feature 추출

1. 구매하는 제품의 다양성

- 1) 고객 별로 구매한 상품들의 고유한 수를 계산하기
- 2) `user_rfm` 테이블과 결과를 합치기
- 3) `user_data` 라는 이름의 테이블에 저장하기

```
CREATE OR REPLACE TABLE `project-1799b89a-43b7-4dae-ac9.modulabs_project.user_data` AS
WITH unique_products AS (
  SELECT
    CustomerID,
    COUNT(DISTINCT StockCode) AS unique_products
  FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.data`
  GROUP BY CustomerID
)
SELECT ur.*, up.* EXCEPT (CustomerID)
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.user_rfm` AS ur
JOIN unique_products AS up
ON ur.CustomerID = up.CustomerID;
```

 이 문으로 이름이 `user_data`인 테이블이 교체되었습니다.

2. 평균 구매 주기

- 고객들의 쇼핑 패턴을 이해하는 것을 목표 (고객 별 재방문 주기 살펴보기)
 - 균 구매 소요 일수를 계산하고, 그 결과를 `user_data` 에 통합

```
CREATE OR REPLACE TABLE `project-1799b89a-43b7-4dae-ac9.modulabs_project.user_data` AS
```

```

WITH purchase_dates AS (
    SELECT DISTINCT
        CustomerID,
        DATE(InvoiceDate) AS InvoiceDay
    FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.dat
a`
),
date_diff_table AS (
    SELECT
        CustomerID,
        DATE_DIFF(
            InvoiceDay,
            LAG(InvoiceDay) OVER (PARTITION BY CustomerID ORDER B
Y InvoiceDay),
            DAY
        ) AS interval_
    FROM purchase_dates
),
avg_cycle AS (
    SELECT
        CustomerID,
        CASE
            WHEN ROUND(AVG(interval_), 2) IS NULL THEN 0
            ELSE ROUND(AVG(interval_), 2)
        END AS average_interval
    FROM date_diff_table
    GROUP BY CustomerID
)
SELECT
    ud.*,
    ac.average_interval
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.user_
data` AS ud
LEFT JOIN avg_cycle AS ac
    ON ud.CustomerID = ac.CustomerID;

```



이 문으로 이름이 user_data인 테이블이 교체되었습니다.

3. 구매 취소 경향성

- 고객의 취소 패턴 파악하기

- 1) 취소 빈도(cancel_frequency) : 고객 별로 취소한 거래의 총 횟수

- 2) 취소 비율(cancel_rate) : 각 고객이 한 모든 거래 중에서 취소를 한 거래의 비율

- 취소 빈도와 취소 비율을 계산하고 그 결과를 `user_data`에 통합하기
(취소 비율은 소수점 두번째 자리)

```
CREATE OR REPLACE TABLE `project-1799b89a-43b7-4dae-ac9.modulabs_project.user_data` AS

WITH TransactionInfo AS (
  SELECT
    CustomerID,
    COUNT(DISTINCT InvoiceNo) AS total_transactions,
    COUNT(DISTINCT CASE WHEN InvoiceNo LIKE 'C%' THEN InvoiceNo END) AS cancel_frequency
  FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.data`
  GROUP BY CustomerID
)

SELECT
  u.*,
  t.* EXCEPT(CustomerID),
  ROUND(t.cancel_frequency / t.total_transactions, 2) AS cancel_rate
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.user_data` AS u
LEFT JOIN TransactionInfo AS t
ON u.CustomerID = t.CustomerID;
```



이 문으로 이름이 user_data인 테이블이 교체되었습니다.

- 다양한 컬럼들을 활용하여 고객의 구매 패턴과 선호도를 보다 심층적으로 이해할 수 있도록 최종적으로 `user_data` 를 출력하기

```
SELECT *
FROM `project-1799b89a-43b7-4dae-ac9.modulabs_project.user_data`;
```

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_products	average_interval	total_transactions	cancel_frequency	cancel_rate
1	17553	1	1048	129	1487.6	1487.6	2	0.0	1	0	0.0
2	14371	1	34	295	100.5	100.5	4	0.0	1	0	0.0
3	17022	1	108	31	71.0	71.0	7	0.0	1	0	0.0
4	13182	1	201	45	121.2	121.2	11	0.0	1	0	0.0
5	18262	1	182	140	149.5	149.5	13	0.0	1	0	0.0
6	15504	1	33	81	52.8	52.8	13	0.0	1	0	0.0
7	15699	1	113	17	309.7	309.7	18	0.0	1	0	0.0

회고

KEEP :

결측치 → 중복값 → 이상값 순서로 체계적으로 전처리를 진행했다

평소에 모르고 그냥 넘어갔을 수도 있을 법한 부분도 세세하게 전처리 하는 법을 배울 수 있었다

특히 효과적이었던 부분:

- `COUNT`, `DISTINCT`, `GROUP BY` 로 데이터 구조를 먼저 파악한 뒤 전처리 진행
- `CustomerID`, `Description` 의 결측 비율을 확인하고 분석 목적에 맞는 삭제 전처리 전략 선택
- `StockCode` 의 숫자 개수 분석으로 `POST` 같은 비상품 코드 이상치 가능성 탐지
- 단순 값 제거가 아니라 왜 제거해야 하는지 데이터 맥락을 먼저 이해하고 판단

PROBLEM :

원본 **data** 테이블이 달라져 있어서 data 파일을 재업로드 후에 전처리 과정을 다시 수행했다

- 전처리 단계를 처음부터 다시 수행하는 비효율 발생
-

Try :

결측치 처리 → 중복 제거 → 이상치 탐지 순서를 사전에 계획하고 SQL을 실행하여 불필요한 재작업을 방지

자연어 → 쿼리, 쿼리 → 자연어로 바꾸는 연습 및 실행 순서 암기

쿼리 실행에 그치지 않고 왜 해당 데이터를 제거하는지와 분석 결과에 미치는 영향을 함께 확인